A MINI PROJECT REPORT

ON

SOCIAL NETWORK FRIEND RECOMMENDATION SYSTEM USING GRAPH ALGORITHMS

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING / TECHNOLOGY

IN

[COMPUTER SCIENCE AND ENGINEERING]

Submitted by

NAMES:[E.VISHNU,D.SRAVAN KUMAR,U.VENU]

Roll Nos: [23AK1A05N3,23AK1A05J5,23AK1A05M9]

[ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES,TIRUPATI]

[JNTU ANANTAPUR]

Academic Year: 2025 – 2026

CERTIFICATE

This is to certify that the mini project report entitled “Social Network Friend Recommendation System Using Graph Algorithms” is a bonafide work carried out by [E.VISHNU,D.SRAVAN KUMAR,U.VENU], bearing Roll Number [23AK1A05N3,23AK1A05J5,23AK1A05M9], students of [COMPUTER SCIENCE AND ENGINEERING] at [ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ,TIRUPATI], in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering/Technology during the academic year 2025–2026.

The matter embodied in this project report has not been submitted to any other University or Institute for the award of any degree or diploma. The work has been carried out under my supervision and guidance, and the project has reached the standard of fulfilling the requirements as per the academic regulations of the institute.

[T.SREENIVASULA REDDY]

Head of Department

External Examiner

Date: _____ Place: _____

DECLARATION

I,[E.VISHNU,D.SRAVANKUMAR,U.VENU],bearingRollNumber [23AK1A05N3,23AK1A05J5,

23AK1A05M9],hereby declare that the mini project report titled “Social Network Friend Recommendation System Using Graph Algorithms” submitted to [ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES TIRUPATI], [UNIVERSITY NAME], is the result of original work carried out by us.

I further declare that this project work is the outcome of my own efforts and that it has not been submitted, either in part or in full, to any other University or Institution for the award of any degree, diploma or similar qualification. All sources of information used in this work have been duly acknowledged.

Place: _____ NAMES:E.VISHNU,D.SRAVANKUMAR,U.VENU

Date: _____ RollNo:[23AK1A05N3,23AK1A05J5,23AK1A05M9]

ACKNOWLEDGEMENT

The successful completion of any task is incomplete without acknowledging the people who made it possible. I take this opportunity to express my sincere gratitude to all those who supported me throughout the duration of this mini project.

First and foremost, I wish to express my deep sense of gratitude to my project guide [GUIDE NAME], Department of [COMPUTER SCIENCE AND ENGIONEERING], [ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ,TIRUPATI], for the constant supervision, valuable suggestions, technical guidance and continuous encouragement extended throughout the course of this work. Their expert knowledge and patience were instrumental in shaping this project.

I sincerely thank [T.SREENIVASULA REDDY], Head of the Department of [COMPUTER SCIENCE AND ENGINEERING], for providing the necessary infrastructure, laboratory facilities and a healthy academic environment to carry out this project.

I also wish to express my gratitude to the Principal of [ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ,TIRUPATI] and the entire faculty of the department for their kind support, motivation and constructive feedback during the various phases of project development.

I am also thankful to my friends and classmates for their valuable suggestions and discussions which helped me to refine various modules of the project. Finally, I express my heartfelt thanks to my parents and family members for their unconditional love, blessings and moral support during my academic journey.

NAMES:[E.VISHNU,D.SRAVAN KUMAR,U.VENU**]**
RollNo: [23AK1A05N3,23AK1A05J5,23AK1A05M9]

ABSTRACT

Online social networks have transformed the way people interact, communicate and share information. As the number of users in such networks grows exponentially, helping users discover new connections that are relevant and meaningful becomes an essential feature. A well-designed friend recommendation system not only enhances user engagement but also increases the overall value of the network by enabling stronger community formation.

This project, titled “Social Network Friend Recommendation System Using Graph Algorithms”, proposes and implements a full-stack web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js) that models the social network as a graph data structure. Each user is represented as a node and each friendship as an undirected edge. The system stores user data in MongoDB using an adjacency-list style schema for efficient traversal.

The recommendation engine combines two graph-based techniques. The first technique computes mutual friends between any two users and uses the count of shared connections as a similarity score to rank suggestions. The second technique applies a Breadth-First Search (BFS) traversal starting from the logged-in user to discover users that are exactly two hops away (friends of friends), excluding direct friends and the user themselves. The combination of these two strategies produces meaningful and explainable recommendations.

The application provides user registration and secure login using JSON Web Tokens (JWT) and bcrypt password hashing, profile management, sending and accepting friend requests, viewing mutual friends, searching for users and receiving real-time friend suggestions. The frontend, developed in React.js, offers a clean and responsive interface while the backend, built using Node.js and Express.js, exposes a set of RESTful APIs.

Testing has been performed at the unit and integration level using tools such as Postman and Jest. The proposed system demonstrates that graph-based algorithms can deliver fast, accurate and explainable friend recommendations even on moderately large datasets, and lays a strong foundation for future enhancements such as machine learning based ranking, interest-based suggestions and real-time notifications.

Keywords: Social Network, Friend Recommendation, Graph Algorithms, Breadth-First Search, Mutual Friends, MERN Stack, MongoDB, JWT.

TABLE OF CONTENTS

No.	Title	Page
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	List of Figures	v
	List of Tables	vi
1	Introduction	1
2	Literature Survey	5
3	Existing System	7
4	Proposed System	9
5	System Requirements	11
6	System Analysis	13
7	System Design	15
8	Modules Description	19
9	Graph Algorithm Explanation	22
10	Implementation Details	26
11	Testing	29
12	Results and Screenshots	31
13	Advantages	33
14	Limitations	34
15	Future Enhancements	35
16	Conclusion	36
	References	37
	Appendix	38

LIST OF FIGURES

Fig. No.	Description	Page
7.1	System Architecture Diagram	15
7.2	Use Case Diagram	16
7.3	Data Flow Diagram (Level 0 and Level 1)	17
7.4	Entity-Relationship (ER) Diagram	18
9.1	Graph Representation of Users and Friendships	22
9.2	BFS Traversal for Friend-of-Friend Suggestion	24
12.1	Login Page Screenshot	31
12.2	Registration Page Screenshot	31
12.3	Dashboard Screenshot	32
12.4	User Profile Page Screenshot	32
12.5	Friend Requests Page Screenshot	32
12.6	Friend Recommendation Page Screenshot	33
12.7	Mutual Friends Page Screenshot	33

LIST OF TABLES

Table No.	Description	Page
5.1	Hardware Requirements	11
5.2	Software Requirements	12
7.1	User Collection Schema	18
7.2	FriendRequest Collection Schema	19
9.1	Adjacency List Example	23
11.1	Test Cases	30
A.1	API Endpoints	38

CHAPTER 1: INTRODUCTION

1.1 Background

Online Social Networking Sites (OSNs) such as Facebook, Instagram, LinkedIn and Twitter have become an indispensable part of everyday communication for billions of users across the world. These platforms allow individuals to create personal profiles, share content, exchange messages and most importantly, form digital connections with other users. The strength of any social network lies in its ability to keep its users engaged, and one of the most powerful drivers of engagement is the ease with which users can discover and connect with new people.

A social network is naturally modelled as a graph data structure. The users of the network correspond to the vertices (nodes) of the graph, and the friendships or follow relationships correspond to the edges. Because friendships in platforms like Facebook are bidirectional, an undirected graph representation is appropriate. This graph-theoretic view enables the use of wellstudied algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), shortest path computations and community detection to extract meaningful insights from the network.

Among the many features offered by modern social platforms, the friend suggestion or “People You May Know” feature is one of the most widely used. It analyses the structure of the social graph along with various other signals such as common interests, mutual friends, geographical location and interaction patterns to recommend potential connections. This project focuses primarily on the structural component of such a recommendation system, namely mutual friends and breadth-first traversal.

1.2 Problem Statement

Although large-scale platforms employ sophisticated proprietary recommendation engines, building a transparent, lightweight and educational friend recommendation system based purely on graph algorithms is a non-trivial task. Many existing academic implementations either focus only on theoretical algorithms without an end-to-end web application, or build full web applications without explaining the underlying graph logic clearly.

The problem addressed by this project is therefore the design and implementation of a complete, full-stack social network application in which the recommendation engine uses well-defined

graph algorithms (mutual friends and BFS) and where every component, from the database schema to the frontend interface, is built around the graph abstraction.

1.3 Need for the System

The proposed system is needed for the following reasons:

- Manually searching for people to connect with in a large network is inefficient and rarely produces meaningful results.
- Recommendation systems based on mutual friends and graph traversal provide explainable suggestions, which increases user trust compared to opaque black-box models.
- Educational institutions and learners benefit from a working reference implementation that demonstrates graph theory in a real-world full-stack application.
- Small-scale community platforms (college groups, alumni networks, hobby communities) require lightweight recommendation features that can be hosted without large infrastructure costs.

1.4 Objectives

1. To design and develop a full-stack web application using the MERN stack that models a social network as a graph.
2. To implement secure user authentication and authorization using JWT and bcrypt.
3. To allow users to send, accept and reject friend requests, view their friend list and search for other users.
4. To implement a recommendation engine that suggests friends based on the count of mutual connections.
5. To implement a Breadth-First Search based recommendation that suggests users at a graph distance of exactly two from the current user.
6. To evaluate the correctness and performance of the system using unit testing and integration testing.

1.5 Scope

The scope of this project covers the design, implementation and testing of a web-based social network friend recommendation system. The system supports the following functionality: user registration and login, profile creation and editing, sending and managing friend requests, friend listing, mutual friend display, user search, and friend recommendations based on graph algorithms.

The system is intended as a single-page web application accessible through modern web browsers. It does not aim to replace large commercial platforms but to serve as a demonstrable and extensible academic prototype. Advanced features such as messaging, content sharing, video calling and machine learning based personalization are outside the scope of the current version and are listed as future enhancements.

CHAPTER 2: LITERATURE SURVEY

A literature survey was conducted to study existing approaches to friend recommendation in online social networks. The relevant work can broadly be grouped into three categories: structural (graphbased) approaches, content-based approaches and hybrid approaches.

2.1 Graph-Based Approaches

Structural approaches use only the topology of the social graph. Early works by Liben-Nowell and Kleinberg (2007) on the link prediction problem in social networks evaluated several similarity measures, including Common Neighbours, Jaccard Coefficient, Adamic/Adar Index and Katz Index. They concluded that even simple measures like Common Neighbours perform remarkably well on large real-world networks. The mutual-friends technique used in this project is a direct application of the Common Neighbours measure.

Breadth-First Search has long been used to enumerate friends-of-friends and is the basis of many practical implementations. Its linear time complexity in terms of the number of vertices and edges makes it especially suitable for real-time suggestion features.

2.2 Content-Based Approaches

Content-based approaches recommend users based on shared attributes such as interests, education, location, age group or profession. They are useful when two users have little or no overlap in their immediate networks but share strong demographic or interest signals. While powerful, they require richer profile data and careful handling of user privacy.

2.3 Hybrid Approaches

Modern commercial systems use hybrid approaches that combine graph signals, content similarity and behavioural signals (such as profile visits, message exchange, and tag co-occurrence). These systems are often powered by machine learning models such as collaborative filtering, matrix factorization and graph neural networks. While such approaches achieve higher accuracy, they require significant computational resources and large training datasets, which makes them unsuitable for small academic projects.

2.4 Summary of Survey

Based on the survey, it is evident that graph-based techniques offer a strong baseline that is computationally cheap, easy to implement and produces explainable results. Therefore, this project adopts a graph-first approach using mutual friends and BFS, while keeping the architecture extensible enough to incorporate content-based and hybrid techniques in future iterations.

CHAPTER 3: EXISTING SYSTEM

Existing online social networks such as Facebook, LinkedIn and Instagram already provide friend or connection suggestions through features like “People You May Know”, “Suggested Connections” or “Suggested for You”. These features are powered by large-scale recommendation engines that combine multiple signals including mutual friends, contact list uploads, work and education history, interaction patterns, photo tags and machine learning models.

However, these recommendation engines are proprietary and their inner workings are not publicly available. Smaller community platforms and academic projects therefore cannot directly reuse them. Many open-source social network templates either do not include a recommendation feature at all, or implement it as a simple random suggestion that does not consider the structure of the network.

3.1 Limitations of Existing System

- Commercial recommendation engines are proprietary black boxes; the logic behind a suggestion is not transparent to the user.
- Most existing open-source MERN social network templates do not include any graphbased recommendation logic.
- Naïve approaches such as suggesting random users or all unconnected users provide poor user experience and do not scale.
- Recommendation features that depend on external data sources (contact list, browsing history) raise serious privacy concerns.
- Many existing systems do not display mutual friends along with the suggestion, which reduces user trust in the recommendation.
- Heavy machine learning based systems require expensive infrastructure and large training datasets, which are not feasible for small projects.

CHAPTER 4: PROPOSED SYSTEM

The proposed system is a full-stack social network web application built on the MERN stack that performs friend recommendations using transparent, well-understood graph algorithms. The

social network is modelled as an undirected graph where users are vertices and friendships are edges. The graph is stored in MongoDB using an adjacency-list style representation in which every user document contains an array of references to its direct friends.

Two complementary algorithms power the recommendation engine. The first algorithm computes mutual friends between the logged-in user and every other user in the network and ranks the candidates by the number of mutual friends. The second algorithm performs a Breadth-First Search starting at the logged-in user and collects all users at a graph distance of exactly two, which corresponds to friends-of-friends. The final list of suggestions is the intersection of these two strategies, sorted by mutual-friend count.

In addition to recommendations, the system provides standard social-network features such as registration, login, profile management, friend request handling, friend list display, mutual friend display and user search. Security is enforced using JSON Web Tokens for stateless authentication and bcrypt for password hashing.

4.1 Advantages of Proposed System

- Recommendations are explainable: every suggestion is accompanied by the count and the list of mutual friends.
- Algorithms used are well-known, taught in standard data structures courses, and easy to verify.
- The system is lightweight and can be hosted on free-tier cloud platforms with minimal cost.
- The MERN stack provides a uniform JavaScript codebase across the frontend and backend, which simplifies development and maintenance.
- MongoDB's flexible schema makes it easy to extend the user document with additional attributes such as interests or location in the future.
- JWT-based authentication ensures stateless, scalable and secure access to all API endpoints.
- The architecture cleanly separates the recommendation logic from the rest of the application, allowing future replacement with machine learning models.

CHAPTER 5: SYSTEM REQUIREMENTS

5.1 Hardware Requirements

The minimum hardware configuration required to develop and run the system is described in Table

5.1.

Component	Specification
Processor	Intel Core i3 (8th Generation) or equivalent / higher
RAM	4 GB minimum, 8 GB recommended
Hard Disk	256 GB SSD or 500 GB HDD
Display	13-inch or larger, 1366 × 768 resolution
Network	Stable broadband internet connection
Input Devices	Standard keyboard and mouse

5.2 Software Requirements

The software stack used in the development of this project is summarized in Table 5.2.

Component	Specification
Operating System	Windows 10/11, Linux (Ubuntu 22.04+) or macOS
Frontend Framework	React.js (v18 or above)
Backend Runtime	Node.js (v18 LTS or above)
Web Framework	Express.js (v4)
Database	MongoDB (v6) – local or MongoDB Atlas
Authentication	JSON Web Token (jsonwebtoken), bcrypt / bcryptjs
Web Browser	Google Chrome, Mozilla Firefox or Microsoft Edge (latest)
IDE / Editor	Visual Studio Code
API Testing	Postman

Version Control	Git and GitHub
Package Manager	npm or yarn

CHAPTER 6: SYSTEM ANALYSIS

6.1 Functional Requirements

Functional requirements describe the specific behaviour or functions of the system. The proposed system shall provide the following functions:

1. The system shall allow new users to register by providing a name, email address and password.
2. The system shall validate user input and ensure email uniqueness during registration.
3. The system shall securely hash all passwords using bcrypt before storing them in the database.
4. The system shall allow registered users to log in using email and password and issue a JWT on successful authentication.
5. The system shall allow authenticated users to view and edit their profile information.
6. The system shall allow users to search for other users by name or email.
7. The system shall allow users to send friend requests to other users.
8. The system shall allow recipients of friend requests to accept or reject them.
9. The system shall display the list of accepted friends for the logged-in user.
10. The system shall compute and display the count and the list of mutual friends between any two users.
11. The system shall generate friend recommendations using mutual friends and BFS traversal.
12. The system shall provide a logout functionality that invalidates the client-side token.

6.2 Non-Functional Requirements

- Performance: API responses for recommendation requests shall be returned within 2 seconds for networks with up to 10,000 users.
- Scalability: The architecture shall support horizontal scaling of the backend through stateless JWT authentication.

- Security: All passwords shall be hashed using bcrypt with a salt round of at least 10. All protected endpoints shall validate the JWT before processing.
- Usability: The user interface shall be clean, responsive and accessible on both desktop and mobile browsers.
- Reliability: The system shall handle invalid inputs and unexpected errors gracefully without crashing.
- Maintainability: The code shall follow modular structure with clear separation between routes, controllers, services and models.
- Portability: The system shall be deployable on any platform that supports Node.js and MongoDB.

CHAPTER 7: SYSTEM DESIGN

7.1 System Architecture

The system follows the standard three-tier architecture consisting of a presentation tier, an application tier and a data tier. The presentation tier is a Single Page Application built with React.js that runs in the user's browser. The application tier consists of a Node.js server using the Express.js framework, which exposes a set of RESTful APIs and implements business logic including the recommendation engine. The data tier is a MongoDB database that persists user, friendship and request data.

Communication between the presentation tier and the application tier happens over HTTP using JSON payloads. The application tier communicates with MongoDB through the Mongoose Object Data Modelling (ODM) library. Authentication is implemented using JSON Web Tokens, which are stored in the browser's localStorage on the client side and sent with every protected request in the Authorization header.

[Figure 7.1: System Architecture Diagram – Browser ↔ React Frontend ↔ Express REST API ↔ MongoDB]

7.2 Use Case Diagram

The use case diagram identifies the actors and the use cases supported by the system. There are two primary actors: the User (a registered member of the social network) and the System (the backend services). The main use cases include Register, Login, View Profile, Edit Profile, Search Users, Send Friend Request, Accept/Reject Friend Request, View Friends, View Mutual Friends, View Recommendations and Logout.

Each use case has an associated precondition (for example, the user must be authenticated to access most of the use cases) and a postcondition (for example, a friend request must be persisted after it has been sent). The diagram shows the User actor connected to each use case via an association line, and inheritance / inclusion relationships where appropriate.

[Figure 7.2: Use Case Diagram showing actors and use cases]

7.3 Data Flow Diagram

The Data Flow Diagram (DFD) graphically represents the flow of data through the system. The Level 0 DFD (Context Diagram) shows the entire system as a single process labelled “Friend Recommendation System” which interacts with the external User entity. The User sends requests such as login credentials, profile updates, friend requests and search queries to the system and receives responses such as JWT tokens, profile data, friend lists and recommendations.

The Level 1 DFD decomposes the single process into sub-processes: Authentication, Profile Management, Friend Request Handling, Recommendation Engine and Search. Each sub-process reads from and writes to the appropriate data stores: Users, Friendships and FriendRequests. Arrows in the diagram represent the direction of data flow between sub-processes and data stores.

[Figure 7.3: Level 0 and Level 1 Data Flow Diagrams]

7.4 Entity-Relationship Diagram

The Entity-Relationship (ER) Diagram models the conceptual structure of the database. The principal entities are User and FriendRequest. The User entity has attributes such as _id, name, email, passwordHash, bio, profilePicture, friends (an array of User references) and createdAt. The FriendRequest entity has attributes _id, sender (User reference), receiver (User reference), status (pending/accepted/rejected) and createdAt.

The relationship between User and User through the friends array is a many-to-many selfreferential relationship that effectively represents the edges of the social graph. The relationship between User and FriendRequest is a one-to-many relationship: one user can send or receive many requests.

[Figure 7.4: Entity-Relationship Diagram]

7.5 Database Design

Two main MongoDB collections are used: users and friendrequests. Table 7.1 describes the schema of the User collection and Table 7.2 describes the FriendRequest collection.

Table 7.1: User Collection Schema

Field	Type	Description
_id	ObjectId	Primary key generated by MongoDB
name	String	Full name of the user
email	String	Unique email address used for login
passwordHash	String	Bcrypt hash of the user's password
bio	String	Short description about the user
profilePicture	String	URL of the profile picture
friends	Array<ObjectId>	References to user documents that are friends
createdAt	Date	Timestamp when the account was created

Table 7.2: FriendRequest Collection Schema

Field	Type	Description
_id	ObjectId	Primary key
sender	ObjectId (User)	User who sent the request
receiver	ObjectId (User)	User who received the request
status	String (enum)	pending / accepted / rejected
createdAt	Date	Timestamp when the request was created

CHAPTER 8: MODULES DESCRIPTION

The application has been divided into the following logical modules. Each module is responsible for a well-defined set of features and is implemented as a combination of frontend components and backend routes/controllers.

8.1 User Authentication Module

The authentication module manages user registration, login and session handling. During registration, the password supplied by the user is hashed using bcrypt with a salt round of 10 before being stored in the database. On successful login, the server generates a JSON Web Token signed with a secret key and returns it to the client. The client stores the token in localStorage and includes it in the Authorization header of every subsequent request. A custom Express middleware validates the token on every protected route.

8.2 User Profile Module

This module is responsible for displaying and updating user profile information. Users can update their name, bio and profile picture. Profile information of other users (excluding the password hash) can be viewed by any authenticated user. The frontend uses controlled React components and form validation to ensure that updates are well-formed before being submitted to the backend.

8.3 Friend Request Module

This module handles the lifecycle of a friend request. A user can send a request to another user, and the receiver can either accept or reject it. When a request is accepted, both users' friends arrays are updated atomically, effectively adding an undirected edge in the social graph. Pending requests are listed in a dedicated section on the dashboard so that users can act on them quickly. Duplicate requests and requests to oneself are prevented at the backend.

8.4 Friend Recommendation Module

The recommendation module is the core of the project. It exposes an API endpoint that, given the logged-in user, returns a ranked list of suggested users. The module uses both mutual friend

analysis and BFS traversal as described in Chapter 9. The result is returned as a JSON array containing the suggested user's basic information along with the number of mutual friends, allowing the frontend to render explainable recommendations.

8.5 Mutual Friends Module

This module computes and displays the mutual friends between the logged-in user and any other selected user. Mutual friends are determined by computing the intersection of the friends arrays of both users. The result is presented to the user along with profile photos and links, which encourages further exploration of the social graph.

8.6 Search Users Module

The search module allows users to find other users by typing a name or part of an email address. The backend performs a case-insensitive regular expression search on the name and email fields of the users collection and returns matching results. Pagination is supported to keep response sizes manageable for large networks.

CHAPTER 9: GRAPH ALGORITHM EXPLANATION

9.1 Graph Representation

The social network is modelled as an undirected graph G = (V, E), where V is the set of users (vertices) and E is the set of friendships (edges). Because friendship is bidirectional (if A is a friend of B, then B is also a friend of A), the graph is undirected. Self-loops and multiple edges between the same pair of vertices are not allowed.

In this project, the graph is stored using an adjacency-list representation. Each user document in MongoDB contains a field friends which is an array of ObjectIds referring to the user's direct neighbours in the graph. This representation is well suited for sparse graphs, which social networks are known to be, because it consumes O(V + E) space rather than the $O(V^2)$ space required by an adjacency matrix.

Table 9.1: Adjacency List Example

User	Friends (Adjacency List)
A	B, C, D
B	A, C, E
C	A, B, E, F
D	A, F
E	B, C
F	C, D

[Figure 9.1: Graph Representation of Users as Nodes and Friendships as Edges]

9.2 Mutual Friends Recommendation

The mutual friends algorithm is based on the Common Neighbours similarity measure. For any two users u and v, the mutual friend count is defined as $|N(u) \cap N(v)|$, where N(x) denotes the set of direct friends of user x. Users with a higher mutual friend count are more likely to be relevant suggestions.

Algorithm: Mutual Friends Recommendation

```
INPUT: currentUser u, all users U
OUTPUT: ranked list of suggestions for u
1.      friendsU  ← set of direct friends of u
2.      suggestions ← empty map (user -> mutualCount)
3.      for each user v in U do
4.      if v = u or v ∈ friendsU then continue
5.      friendsV ← set of direct friends of v
6.      common  ← friendsU ∩ friendsV
7.      if |common| > 0 then
8.      suggestions[v] ← |common|
9.      return suggestions sorted by value DESC
```

The intersection operation in step 6 can be computed efficiently using hash sets in O(min(|N(u)|, | N(v)|)).

9.3 BFS-Based Friend Suggestion

Breadth-First Search is used to systematically explore the graph in layers starting from the source node. In the context of friend suggestion, BFS is used to find all users at a graph distance of exactly two from the current user. These users correspond to “friends of friends” and are typical candidates for recommendation.

Algorithm: BFS-based Friend Suggestion

```
INPUT: currentUser u
OUTPUT: set of users at distance 2 from u
1.      visited ← { u } ∪ N(u)       // do not suggest u or direct friends
2.      queue   ← empty FIFO queue
3.      enqueue (u, 0) into queue
4.      result  ← empty set
5.      while queue is not empty do
6.      (node, depth) ← dequeue queue
7.      if depth = 2 then
8.      result.add(node)
9.      continue
10.     for each neighbour w of node do
11.     if w ∉ visited then
```

```
12.         visited.add(w)
13.         enqueue (w, depth + 1) into queue
14.     return result
```

[Figure 9.2: BFS Traversal Producing Friend-of-Friend Suggestions]

9.4 Time and Space Complexity

The time complexity of the mutual friends algorithm is O(V × d) where V is the number of users and d is the average degree (average number of friends per user). Since social networks are typically sparse, d is small compared to V, and the algorithm scales well.

The time complexity of the BFS algorithm is O(V + E) in the worst case. However, since the search is restricted to depth 2, the practical running time is much smaller and proportional to the size of the two-hop neighbourhood of the current user.

The space complexity of both algorithms is O(V) due to the visited set and the queue used during traversal. The adjacency list itself consumes O(V + E) space.

CHAPTER 10: IMPLEMENTATION DETAILS

10.1 Frontend Implementation

The frontend is implemented as a Single Page Application using React.js. React Router is used for client-side navigation between pages such as Login, Register, Dashboard, Profile, Friend Requests and Recommendations. Axios is used to make HTTP requests to the backend APIs. The state of the application (current user, friends, recommendations) is managed using React's Context API and the useReducer hook for predictable updates.

Each major view is implemented as a functional React component. Reusable components such as UserCard, FriendRequestCard and RecommendationCard are placed in a shared components folder. Styling is done using CSS modules and a small set of utility classes inspired by modern design systems. The application is responsive and works well on both desktop and mobile browsers.

10.2 Backend API Implementation

The backend is implemented using Node.js and Express.js. The application is organized into routes, controllers, services and models. Mongoose is used as the ODM library for interacting with MongoDB. The main API endpoints are summarized in Table A.1 in the Appendix.

All protected routes pass through a JWT verification middleware that extracts the token from the Authorization header, verifies it using the secret key, and attaches the decoded user information to the request object. Error handling is centralized in an Express error-handling middleware that returns consistent JSON error responses.

Sample Express Route

```
// routes/recommendations.js const router = require('express').Router(); const auth =
require('../middleware/auth'); const { getRecommendations } =
require('../controllers/recommendController');


router.get('/', auth, getRecommendations);
module.exports = router;
```

10.3 MongoDB Schema

The User and FriendRequest schemas are defined using Mongoose. The User schema includes a friends field of type [ObjectId] with a reference to the User model itself, which allows population of full friend documents when needed. Email is indexed and marked unique.

```
const userSchema = new mongoose.Schema({   name: { type: String, required: true },   email: { type: String, required: true, unique: true, index: true },   passwordHash: { type: String, required: true },   bio: { type: String, default: '' },   profilePicture: { type: String, default: '' },   friends: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }], }, { timestamps: true });
```

10.4 JWT Authentication Flow

1. The client submits email and password to POST /api/auth/login.
2. The server fetches the user document by email and compares the supplied password with the stored bcrypt hash using bcrypt.compare().
3. On a successful match, the server generates a JWT signed with the secret key, embedding the user's id and email in the payload, with an expiration time of 7 days.
4. The server returns the token along with basic user information to the client.
5. The client stores the token in localStorage and includes it as Authorization: Bearer <token> in every subsequent request.
6. On every protected route, an Express middleware verifies the token using jwt.verify(). If the token is invalid or expired, a 401 Unauthorized response is returned.

CHAPTER 11: TESTING

Testing was carried out at the unit and integration levels to ensure the correctness and reliability of the system. Manual exploratory testing was also performed using web browsers and Postman to validate the user experience.

11.1 Unit Testing

Unit tests were written for the recommendation service functions, the JWT helper utilities and the friend-request controllers. The Jest testing framework was used along with mongodb-memoryserver for an in-memory database, which allowed fast and isolated test execution. Each function was tested independently using sample input data and expected output, and edge cases such as empty friend lists and self-recommendation were explicitly verified.

11.2 Integration Testing

Integration tests were carried out using Supertest by issuing HTTP requests against the running Express application connected to a test database. The full request-response cycle, including authentication middleware, controller logic, database interaction and JSON serialization, was validated end to end. Sample flows tested include registration → login → send friend request → accept → fetch friends → fetch recommendations.

11.3 Test Cases

Table 11.1 summarizes the test cases executed during the testing phase.

TC ID	Module	Input	Expected Output	Actual Output	Status
TC-01	Registration	Valid name, email, password	Account created, redirected to login	As expected	Pass
TC-02	Registration	Duplicate email	Error: email already exists	As expected	Pass
TC-03	Login	Correct credentials	JWT issued, redirected to dashboard	As expected	Pass

TC-04	Login	Wrong password	Error 401 Unauthorized	As expected	Pass

TC-05	Profile	Update bio and save	Bio updated in database and UI	As expected	Pass
TC-06	Friend Request	Send request to another user	Request stored as pending	As expected	Pass
TC-07	Friend Request	Send request to self	Error: cannot send to self	As expected	Pass
TC-08	Friend Request	Accept pending request	Both users become friends	As expected	Pass
TC-09	Mutual Friends	Two connected users	Correct intersection of friends	As expected	Pass
TC-10	Recommendation	User with multiple FoF	Ranked list of suggestions	As expected	Pass
TC-11	Search	Partial name query	Matching users returned	As expected	Pass
TC-12	Auth Middleware	Request without token	401 Unauthorized response	As expected	Pass

CHAPTER 12: RESULTS AND SCREENSHOTS

This chapter showcases the final user interface of the system. Each screenshot represents a key page of the application and demonstrates the successful integration of all modules. Placeholders are provided for the screenshots; they should be replaced with actual captures taken from a running deployment of the system.

[Figure 12.1: Login Page – Email and password form with validation messages]

[Figure 12.2: Registration Page – New user form capturing name, email and password]

[Figure 12.3: Dashboard – Overview of friends, pending requests and quick suggestions]

[Figure 12.4: User Profile Page – View and edit personal information]

[Figure 12.5: Friend Requests Page – List of pending requests with Accept/Reject actions]

[Figure 12.6: Friend Recommendation Page – Ranked suggestions with mutual friend count]

[Figure 12.7: Mutual Friends Page – Common friends between two users]

CHAPTER 13: ADVANTAGES

- Provides explainable friend recommendations based on mutual friends and graph traversal.
- Built entirely on the open-source MERN stack, requiring no expensive licences.
- Uses JWT-based stateless authentication, which scales horizontally without sticky sessions.
- Recommendation engine is decoupled from the rest of the application, simplifying future upgrades.
- Schema is flexible and can be extended with interests, location and behavioural data.
- Responsive React frontend offers a smooth user experience on multiple devices.
- Algorithms are well-known and easy to maintain and verify by other developers.

CHAPTER 14: LIMITATIONS

- Recommendations are based only on the structure of the graph; user interests and behaviour are not considered.
- Performance may degrade when computing mutual friends for very large networks (millions of users) without caching.
- The current implementation does not support real-time push notifications for new friend requests or messages.
- No chat or content-sharing feature is included in the current version.
- There is no protection against fake profiles or spam friend requests at the algorithmic level.
- Profile pictures are stored as URLs and there is no built-in file upload service.

CHAPTER 15: FUTURE ENHANCEMENTS

The system has been designed to be extensible. The following enhancements are proposed for future iterations:

1. Machine Learning Based Recommendations: A collaborative filtering or graph neural network model could be trained on the interaction logs to produce more accurate and personalized recommendations.
2. Common Interests Based Suggestions: Users could be tagged with interests such as music, sports or technology, and recommendations could be ranked using the Jaccard similarity of interests in addition to mutual friends.
3. Location-Based Friend Suggestions: Users in the same geographical area (city, college, workplace) could be prioritized in the suggestion list using geospatial indexes provided by MongoDB.
4. Real-Time Notifications using Socket.io: Friend requests, acceptances and new recommendations could be pushed to the client in real time using WebSocket connections.
5. Recommendation Ranking System: A composite scoring function could be implemented that combines mutual friend count, common interests, location proximity and recency of activity to produce an overall rank for each suggestion.
6. Privacy Controls: Granular privacy settings allowing users to control who can see their profile, send them requests or appear in their recommendations.
7. Mobile Application: A React Native mobile client could be developed to share the same backend APIs.

CHAPTER 16: CONCLUSION

The project “Social Network Friend Recommendation System Using Graph Algorithms” has been successfully designed and implemented using the MERN stack. The system models the social network as an undirected graph and provides friend recommendations using two complementary graph algorithms: mutual-friends similarity and Breadth-First Search up to depth two.

The application supports the core features expected from a modern social network, namely user registration, secure authentication using JWT and bcrypt, profile management, friend request handling, mutual friend display, user search and personalized friend recommendations. Comprehensive testing using unit and integration tests has demonstrated the correctness and reliability of the system under various scenarios.

Through this project, a strong practical understanding of graph data structures, graph algorithms, RESTful API design, NoSQL database modelling and full-stack development with React.js and Node.js has been gained. The codebase is modular, extensible and ready to be enhanced with advanced features such as machine learning based recommendations, real-time notifications and content sharing. Overall, the project meets all the objectives defined at the beginning and serves as a strong foundation for further research and development in the domain of social network analytics.

REFERENCES

1. D. Liben-Nowell and J. Kleinberg, “The Link-Prediction Problem for Social Networks”, Journal of the American Society for Information Science and Technology, 2007.
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.
3. R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.
4. MongoDB Inc., MongoDB Documentation, https://www.mongodb.com/docs/, 2024.
5. Node.js Foundation, Node.js Documentation, https://nodejs.org/en/docs/, 2024.
6. Express.js, Express – Fast, unopinionated, minimalist web framework, https://expressjs.com/, 2024.

7. Meta Open Source, React – A JavaScript library for building user interfaces, https://react.dev/, 2024.
8. Auth0, Introduction to JSON Web Tokens, https://jwt.io/introduction, 2024.
9. D. Easley and J. Kleinberg, Networks, Crowds, and Markets: Reasoning About a Highly Connected World, Cambridge University Press, 2010.
10. A. Mislove et al., “Measurement and Analysis of Online Social Networks”, Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement, 2007.

APPENDIX

A.1 API Endpoints

Table A.1 lists the main RESTful API endpoints exposed by the backend.

Method	Endpoint	Description
POST	/api/auth/register	Create a new user account
POST	/api/auth/login	Authenticate user and issue JWT
GET	/api/users/me	Get profile of the logged-in user
PUT	/api/users/me	Update logged-in user's profile
GET	/api/users/search?q=	Search users by name or email
POST	/api/friends/request/:id	Send a friend request
POST	/api/friends/accept/:id	Accept a pending friend request
POST	/api/friends/reject/:id	Reject a pending friend request
GET	/api/friends	Get list of accepted friends
GET	/api/friends/mutual/:id	Get mutual friends with another user
GET	/api/recommendations	Get ranked friend recommendations

A.2 Sample MongoDB User Schema

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },   email: {     type: String, required: true, unique:
true,     lowercase: true, trim: true, index: true,
  },   passwordHash: { type: String, required: true },   bio: { type: String, default: '' },   profilePicture: {
type: String, default: '' },   friends: [{     type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  }],
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

A.3 Important Code Snippets

A.3.1 JWT Authentication Middleware

```
const jwt = require('jsonwebtoken');


module.exports = function auth(req, res, next) {   const header =
req.headers.authorization;   if (!header || !header.startsWith('Bearer ')) {
return res.status(401).json({ error: 'No token provided' });
  }   try {     const token = header.split(' ')[1];     const decoded = jwt.verify(token,
process.env.JWT_SECRET);     req.user = decoded;     next();   } catch (err) {
return res.status(401).json({ error: 'Invalid or expired token' });   }
};
```

A.3.2 Mutual Friends Computation

```
async function mutualFriends(userAId, userBId) {   const [a, b] = await Promise.all([
User.findById(userAId).select('friends'),     User.findById(userBId).select('friends'),
  ]);
  const setA = new Set(a.friends.map(String));   return b.friends.filter(f =>
setA.has(String(f)));
}
```

A.3.3 BFS Friend-of-Friend Recommendation

```
async function recommendFriends(userId) {
```

```
 const user = await User.findById(userId).select('friends');   const visited = new
Set([String(userId)]);   user.friends.forEach(f => visited.add(String(f)));

 const queue = [{ id: String(userId), depth: 0 }];   const candidates =
new Map(); // id -> mutualCount

 while (queue.length) {     const { id, depth } = queue.shift();     if
(depth === 2) {       const mutual = await mutualFriends(userId,
id);       candidates.set(id, mutual.length);       continue;
   }     const node = await User.findById(id).select('friends');     for
(const nb of node.friends) {       const key = String(nb);       if
(!visited.has(key)) {         visited.add(key);         queue.push({ id: key,
depth: depth + 1 });
     }
   }   }   return [...candidates.entries()]
   .sort((a, b) => b[1] - a[1])
   .map(([id, count]) => ({ id, mutualFriends: count }));
```

— END OF REPORT —